Atividade de Laboratório 5 - Python

Código:

```
"""
Laboratório 5: BI e Data Mining com Python (Pandas) - VERSÃO CORRIGIDA
Disciplina: Gestão de Sistemas de Informação
Professor: Eduardo Max Amaro Amaral

CORREÇÃO: Agrupamento por (Data, Vendedor) para simular "carrinhos de
compras"
"""

import pandas as pd
from datetime import datetime

#
==========================================================================
=====
# PARTE 0: PREPARAÇÃO DO AMBIENTE E CARGA DE DADOS
#
==========================================================================
=====

def load_data():
    """Carrega o arquivo CSV de vendas."""
    csv_path = r"vendas_200_registros.csv"
    df = pd.read_csv(csv_path)
    # Converter coluna Data para datetime (formato DD/MM/YYYY)
    df['Data'] = pd.to_datetime(df['Data'], format='%d/%m/%Y')
    return df


#
==========================================================================
=====
# PREPARAÇÃO PARA MINERAÇÃO: FUNÇÃO AUXILIAR E CONSTRUÇÃO DE CARRINHOS
#
==========================================================================
=====

def prepare_data(df):
```

```
    """
    Prepara os dados para mineração agrupando produtos por (Data,
Vendedor).
    Retorna um dicionário onde a chave é (Data, Vendedor) e o valor é
uma lista de produtos.

    JUSTIFICATIVA: Como cada ID_Transacao tem apenas 1 produto,
interpretamos
    um "carrinho de compras" como todas as vendas de um vendedor em um
dia específico.
    """
    # Agrupar por Data e Vendedor
    carrinhos = df.groupby(['Data',
'Vendedor'])['Produto'].apply(list).to_dict()
    N = len(carrinhos)  # Total de "transações" (cesta por
dia/vendedor)
    return carrinhos, N


def contar_ocorrencias(carrinhos_dict, itens_busca):
    """
    Conta quantas transações contêm TODOS os itens da lista
itens_busca.

    Args:
        carrinhos_dict: Dicionário {(Data, Vendedor): [Produtos]}
        itens_busca: Lista de itens a buscar

    Returns:
        Número de transações que contêm todos os itens
    """
    contador = 0
    for produtos in carrinhos_dict.values():
        # Verifica se todos os itens da busca estão na lista de
produtos da transação
        if all(item in produtos for item in itens_busca):
            contador += 1
    return contador


#
==========================================================================
=====
```

```
# PARTE 1: EXERCÍCIOS PRÁTICOS DE OLAP
#
==========================================================================
=====

def exercicio_1_1_drill_down(df):
    """
    Exercício 1.1 - Operação Drill-down (Detalhamento)

    Pergunta: Qual regional teve o maior faturamento total?
    E dentro da regional Sudeste, qual vendedor (Ana ou Diego) teve o
melhor desempenho?
    """
    print("\n" + "="*80)
    print("EXERCÍCIO 1.1 - OPERAÇÃO DRILL-DOWN (DETALHAMENTO)")
    print("="*80)

    # Parte 1: Faturamento total por regional
    faturamento_regional =
df.groupby('Regional')['Valor_Venda'].sum().sort_values(ascending=False
)

    print("\n[1] Faturamento Total por Regional:")
    print(faturamento_regional)
    regional_maior = faturamento_regional.index[0]
    valor_maior = faturamento_regional.iloc[0]
    print(f"\n✓ Regional com maior faturamento: {regional_maior} (R$
{valor_maior:,.2f})")

    # Parte 2: Dentro da Sudeste, faturamento por vendedor
    print("\n[2] Faturamento por Vendedor na Regional SUDESTE:")
    df_sudeste = df[df['Regional'] == 'Sudeste']
    faturamento_vendedor_sudeste =
df_sudeste.groupby('Vendedor')['Valor_Venda'].sum().sort_values(ascendi
ng=False)
    print(faturamento_vendedor_sudeste)

    vendedor_melhor = faturamento_vendedor_sudeste.index[0]
    valor_vendedor = faturamento_vendedor_sudeste.iloc[0]
    print(f"\n✓ Melhor vendedor em Sudeste: {vendedor_melhor} (R$
{valor_vendedor:,.2f})")
```

```python
def exercicio_1_2_slice(df):
    """
    Exercício 1.2 - Operação Slice (Fatiar)

    Pergunta: Dentro da categoria "Acessórios", qual é o produto mais
vendido em valor total?
    """
    print("\n" + "="*80)
    print("EXERCÍCIO 1.2 - OPERAÇÃO SLICE (FATIAR)")
    print("="*80)

    # Filtrar pela categoria Acessórios
    df_acessorios = df[df['Categoria'] == 'Acessórios']

    # Agrupar por produto e somar valores
    faturamento_produtos =
df_acessorios.groupby('Produto')['Valor_Venda'].sum().sort_values(ascen
ding=False)

    print("\n[1] Produtos em Acessórios (ordenados por faturamento
total):")
    print(faturamento_produtos)

    produto_mais_vendido = faturamento_produtos.index[0]
    valor_total = faturamento_produtos.iloc[0]

    print(f"\n✓ Produto mais vendido em Acessórios:
{produto_mais_vendido}")
    print(f"  Faturamento Total: R$ {valor_total:,.2f}")


def exercicio_1_3_dice(df):
    """
    Exercício 1.3 - Operação Dice (Recortar)

    Pergunta: Crie uma visão cruzando Vendedor (Linhas) com Categoria
(Colunas)
    para faturamento apenas nos últimos dias do período (11/03/2026 a
15/03/2026).
    """
    print("\n" + "="*80)
    print("EXERCÍCIO 1.3 - OPERAÇÃO DICE (RECORTAR)")
    print("="*80)
```

```
    # Filtrar pelos últimos dias do período disponível (11/03/2026 a
15/03/2026)
    data_inicio = pd.to_datetime('2026-03-11')
    data_fim = pd.to_datetime('2026-03-15')

    df_semana = df[(df['Data'] >= data_inicio) & (df['Data'] <=
data_fim)]

    print(f"\n[1] Vendas no período de 11/03/2026 a 15/03/2026:")
    print(f"    Total de registros no período: {len(df_semana)}")

    # Criar pivot table: Vendedor (linhas) × Categoria (colunas)
    pivot = pd.pivot_table(
        df_semana,
        values='Valor_Venda',
        index='Vendedor',
        columns='Categoria',
        aggfunc='sum',
        fill_value=0
    )

    print("\n[2] Matriz cruzada: Vendedor (Linhas) × Categoria
(Colunas)")
    print("     Faturamento Total para cada combinação:")
    print(pivot)

    # Adicionar total por linha
    pivot['TOTAL'] = pivot.sum(axis=1)
    print("\n[3] Com totais por Vendedor:")
    print(pivot)


#
==========================================================================
=====
# PARTE 2: EXERCÍCIOS DE MINERAÇÃO (REGRAS DE ASSOCIAÇÃO)
#
==========================================================================
=====

def exercicio_2_1_regra_smartphone_fone(df, carrinhos, N):
    """
```

```
    Exercício 2.1 - Análise da Regra A (Smartphone → Fone Bluetooth)

    Pergunta: Calcule o Suporte e a Confiança para a regra:
    "Quem compra Smartphone, também compra Fone Bluetooth".

    NOTA: Uma "transação" aqui é definida como todas as vendas de um
vendedor em um dia.
    """
    print("\n" + "="*80)
    print("EXERCÍCIO 2.1 - ANÁLISE DA REGRA A: Smartphone → Fone
Bluetooth")
    print("="*80)

    print(f"\n[Dados Base]")
    print(f"  Total de Transações (N): {N}")
    print(f"  (Onde 'transação' = vendas de um vendedor em um dia)")

    # Contar transações com Smartphone
    smartphone_count = contar_ocorrencias(carrinhos, ['Smartphone'])
    print(f"  Transações com Smartphone: {smartphone_count}")

    # Contar transações com Fone Bluetooth
    fone_count = contar_ocorrencias(carrinhos, ['Fone Bluetooth'])
    print(f"  Transações com Fone Bluetooth: {fone_count}")

    # Contar transações com ambos
    ambos_count = contar_ocorrencias(carrinhos, ['Smartphone', 'Fone
Bluetooth'])
    print(f"  Transações com Smartphone AND Fone Bluetooth:
{ambos_count}")

    # Calcular Suporte e Confiança
    suporte_smartphone = smartphone_count / N
    suporte_ambos = ambos_count / N
    confianca = ambos_count / smartphone_count if smartphone_count > 0
else 0

    print(f"\n[Cálculos]")
    print(f"  Support(Smartphone) = {smartphone_count}/{N} =
{suporte_smartphone:.4f} ({suporte_smartphone*100:.2f}%)")
    print(f"  Support(Smartphone AND Fone) = {ambos_count}/{N} =
{suporte_ambos:.4f} ({suporte_ambos*100:.2f}%)")
```

```
    print(f"  Confidence(Smartphone → Fone) =
{ambos_count}/{smartphone_count} = {confianca:.4f}
({confianca*100:.2f}%)")

    print(f"\n✓ RESULTADO REGRA A:")
    print(f"  Suporte: {suporte_ambos:.4f}")
    print(f"  Confiança: {confianca:.4f}")

    return confianca


def exercicio_2_2_regra_tablet_capa(df, carrinhos, N,
confianca_regra_a):
    """
    Exercício 2.2 - Análise da Regra C (Tablet → Capa Tablet)

    Pergunta: Calcule o Suporte e a Confiança para a regra:
    "Quem compra Tablet, também compra Capa Tablet".
    Compare com a Regra A: qual é mais forte?
    """
    print("\n" + "="*80)
    print("EXERCÍCIO 2.2 - ANÁLISE DA REGRA C: Tablet → Capa Tablet")
    print("="*80)

    print(f"\n[Dados Base]")
    print(f"  Total de Transações (N): {N}")
    print(f"  (Onde 'transação' = vendas de um vendedor em um dia)")

    # Contar transações com Tablet
    tablet_count = contar_ocorrencias(carrinhos, ['Tablet'])
    print(f"  Transações com Tablet: {tablet_count}")

    # Contar transações com Capa Tablet
    capa_count = contar_ocorrencias(carrinhos, ['Capa Tablet'])
    print(f"  Transações com Capa Tablet: {capa_count}")

    # Contar transações com ambos
    ambos_count = contar_ocorrencias(carrinhos, ['Tablet', 'Capa
Tablet'])
    print(f"  Transações com Tablet AND Capa Tablet: {ambos_count}")

    # Calcular Suporte e Confiança
    suporte_tablet = tablet_count / N
```

```
    suporte_ambos = ambos_count / N
    confianca = ambos_count / tablet_count if tablet_count > 0 else 0

    print(f"\n[Cálculos]")
    print(f"  Support(Tablet) = {tablet_count}/{N} =
{suporte_tablet:.4f} ({suporte_tablet*100:.2f}%)")
    print(f"  Support(Tablet AND Capa) = {ambos_count}/{N} =
{suporte_ambos:.4f} ({suporte_ambos*100:.2f}%)")
    print(f"  Confidence(Tablet → Capa) = {ambos_count}/{tablet_count}
= {confianca:.4f} ({confianca*100:.2f}%)")

    print(f"\n✓ RESULTADO REGRA C:")
    print(f"  Suporte: {suporte_ambos:.4f}")
    print(f"  Confiança: {confianca:.4f}")

    # Comparação com Regra A
    print(f"\n[COMPARAÇÃO com Regra A (Smartphone → Fone)]")
    print(f"  Confiança Regra A: {confianca_regra_a:.4f}
({confianca_regra_a*100:.2f}%)")
    print(f"  Confiança Regra C: {confianca:.4f}
({confianca*100:.2f}%)")

    if confianca > confianca_regra_a:
        diferenca = confianca - confianca_regra_a
        print(f"\n  ✓ REGRA C É MAIS FORTE: +{diferenca:.4f}
({diferenca*100:.2f}% maior)")
        print(f"    A regra Tablet → Capa Tablet tem maior confiança
que Smartphone → Fone.")
    elif confianca < confianca_regra_a:
        diferenca = confianca_regra_a - confianca
        print(f"\n  ✗ REGRA C É MAIS FRACA: -{diferenca:.4f}
({diferenca*100:.2f}% menor)")
        print(f"    A regra Tablet → Capa Tablet tem menor confiança
que Smartphone → Fone.")
    else:
        print(f"\n  = REGRAS TÊM FORÇA IGUAL")


#
==========================================================================
=====
# FUNÇÃO PRINCIPAL
```

```
#
==========================================================================
=====

def main():
    """Orquestra a execução de todos os exercícios."""
    print("\n")
    print("╔" + "="*78 + "╗")
    print("│" + " "*78 + "│")
    print("│" + "  LABORATÓRIO 5: BI E DATA MINING COM PYTHON
(PANDAS)".center(78) + "│")
    print("│" + "  Gestão de Sistemas de Informação (VERSÃO
CORRIGIDA)".center(78) + "│")
    print("│" + " "*78 + "│")
    print("╚" + "="*78 + "╝")

    # Carregar dados
    print("\n[INICIALIZANDO...]")
    print("Carregando arquivo de vendas...")
    df = load_data()
    print(f"✓ {len(df)} registros carregados com sucesso!")
    print(f"  Colunas: {', '.join(df.columns.tolist())}")
    print(f"  Período: {df['Data'].min().strftime('%d/%m/%Y')} a
{df['Data'].max().strftime('%d/%m/%Y')}")

    # Preparar dados para mineração
    print("\n[PREPARAÇÃO PARA MINERAÇÃO...]")
    carrinhos, N = prepare_data(df)
    print(f"✓ Estrutura de carrinhos criada!")
    print(f"  Total de 'transações' (Data + Vendedor): {N}")
    print(f"  Agrupamento: Todas as vendas de um vendedor em um dia = 1
'transação'")

    #
==========================================================================
=
    # EXECUTAR EXERCÍCIOS
    #
==========================================================================
=

    # Parte 1: OLAP
    exercicio_1_1_drill_down(df)
```

```
    exercicio_1_2_slice(df)
    exercicio_1_3_dice(df)

    # Parte 2: Mineração
    confianca_regra_a = exercicio_2_1_regra_smartphone_fone(df,
carrinhos, N)
    exercicio_2_2_regra_tablet_capa(df, carrinhos, N,
confianca_regra_a)

    # Finalização
    print("\n" + "="*80)
    print("FIM DA EXECUÇÃO - TODOS OS EXERCÍCIOS COMPLETADOS")
    print("="*80 + "\n")


if __name__ == "__main__":
    main()
```

Execução:

```
╔═══════════════════════════════════════════════════════════════════════════╗
╚═══════════════════════════════════════════════════════════════════════════╝

[INICIALIZANDO...]
Carregando arquivo de vendas...
✓ 200 registros carregados com sucesso!
  Colunas: ID_Transacao, Data, Regional, Vendedor, Categoria, Produto, Valor_Venda
  Período: 01/03/2026 a 15/03/2026

[PREPARAÇÃO PARA MINERAÇÃO...]
✓ Estrutura de carrinhos criada!
  Total de 'transações' (Data + Vendedor): 67
  Agrupamento: Todas as vendas de um vendedor em um dia = 1 'transação'

===============================================================================
EXERCÍCIO 1.1 - OPERAÇÃO DRILL-DOWN (DETALHAMENTO)
===============================================================================

[1] Faturamento Total por Regional:
Regional
Sudeste     107000.0
Sul          88200.0
Nordeste     73450.0
Name: Valor_Venda, dtype: float64

✓ Regional com maior faturamento: Sudeste (R$ 107,000.00)

[2] Faturamento por Vendedor na Regional SUDESTE:
Vendedor
Carla    35450.0
Diego    25450.0
Ana      20400.0
Elena    14250.0
Bruno    11450.0
Name: Valor_Venda, dtype: float64

✓ Melhor vendedor em Sudeste: Carla (R$ 35,450.00)

===============================================================================
EXERCÍCIO 1.2 - OPERAÇÃO SLICE (FATIAR)
===============================================================================

[1] Produtos em Acessórios (ordenados por faturamento total):
Produto
Teclado Mecânico    5250.0
Fone Bluetooth      2000.0
Mouse Gamer         1500.0
Capa Tablet         1300.0
Name: Valor_Venda, dtype: float64
```

```
✓ Produto mais vendido em Acessórios: Teclado Mecânico
  Faturamento Total: R$ 5,250.00

================================================================================
EXERCÍCIO 1.3 - OPERAÇÃO DICE (RECORTAR)
================================================================================

[1] Vendas no período de 11/03/2026 a 15/03/2026:
    Total de registros no período: 62

[2] Matriz cruzada: Vendedor (Linhas) × Categoria (Colunas)
     Faturamento Total para cada combinação:
Categoria  Acessórios  Cozinha  Eletrodomésticos  Eletrônicos  Móveis
Vendedor
Ana            700.0   1800.0            6150.0          0.0     0.0
Bruno          100.0    600.0            4750.0       2500.0  7300.0
Carla         1000.0    600.0           12250.0       6200.0  2900.0
Diego            0.0      0.0           10500.0      10000.0  3300.0
Elena         1200.0   1200.0            4150.0      10800.0     0.0

[3] Com totais por Vendedor:
Categoria  Acessórios  Cozinha  Eletrodomésticos  Eletrônicos  Móveis    TOTAL
Vendedor
Ana            700.0   1800.0            6150.0          0.0     0.0   8650.0
Bruno          100.0    600.0            4750.0       2500.0  7300.0  15250.0
Carla         1000.0    600.0           12250.0       6200.0  2900.0  22950.0
Diego            0.0      0.0           10500.0      10000.0  3300.0  23800.0
Elena         1200.0   1200.0            4150.0      10800.0     0.0  17350.0

================================================================================
EXERCÍCIO 2.1 - ANÁLISE DA REGRA A: Smartphone → Fone Bluetooth
================================================================================

[Dados Base]
  Total de Transações (N): 67
  (Onde 'transação' = vendas de um vendedor em um dia)
  Transações com Smartphone: 6
  Transações com Fone Bluetooth: 10
  Transações com Smartphone AND Fone Bluetooth: 0

[Cálculos]
  Support(Smartphone) = 6/67 = 0.0896 (8.96%)
  Support(Smartphone AND Fone) = 0/67 = 0.0000 (0.00%)
  Confidence(Smartphone → Fone) = 0/6 = 0.0000 (0.00%)

✓ RESULTADO REGRA A:
  Suporte: 0.0000
  Confiança: 0.0000
```

```
================================================================================
EXERCÍCIO 2.2 - ANÁLISE DA REGRA C: Tablet → Capa Tablet
================================================================================

[Dados Base]
  Total de Transações (N): 67
  (Onde 'transação' = vendas de um vendedor em um dia)
  Transações com Tablet: 9
  Transações com Capa Tablet: 11
  Transações com Tablet AND Capa Tablet: 2

[Cálculos]
  Support(Tablet) = 9/67 = 0.1343 (13.43%)
  Support(Tablet AND Capa) = 2/67 = 0.0299 (2.99%)
  Confidence(Tablet → Capa) = 2/9 = 0.2222 (22.22%)

✓ RESULTADO REGRA C:
  Suporte: 0.0299
  Confiança: 0.2222

[COMPARAÇÃO com Regra A (Smartphone → Fone)]
  Confiança Regra A: 0.0000 (0.00%)
  Confiança Regra C: 0.2222 (22.22%)

  ✓ REGRA C É MAIS FORTE: +0.2222 (22.22% maior)
    A regra Tablet → Capa Tablet tem maior confiança que Smartphone → Fone.

================================================================================
FIM DA EXECUÇÃO - TODOS OS EXERCÍCIOS COMPLETADOS
================================================================================
```